Network Programming (IE2102)

2nd Year, 2nd Semester

Assignment

Design and Implementation of a Secure Multiprocessor TCP Application Server with Custom Protocol and Session Management

Submitted to

Sri Lanka Institute of Information Technology

IT24104294

Sumanawardane. W.D.T.N

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

10/04/2026

Declaration

I certify that this report does not incorporate, without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person, except where due reference is made in text.

Contents

Hostname Configuration 3

Network Configuration 3

Server running proof 4

User Registration 4

User Authentication 5

Authorized Commands 6

User log-out 8

User data storage 9

Handling Multiple Clients 10

Build server 11

Figure Table 12

Hostname Configuration

Host name of the machine. `hostnamectl` returns the host name, ID, and architecture of the machine.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ hostnamectl
 Static hostname: DESKTOP-CG383E4
       Icon name: computer-container
         Chassis: container
      Machine ID: 0f8c275aef9a4a789d6f1367b54daafc
         Boot ID: 6e8eca0e940b467ea6864fca4e01584f
  Virtualization: wsl
Operating System: Ubuntu 22.04.4 LTS
          Kernel: Linux 5.15.153.1-microsoft-standard-WSL2
    Architecture: x86-64
```

Figure 1.1: Hostname

Network Configuration

Here is the network interface that this machine has, and we utilize the loopback address to communicate with the server and the client.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:97:0e:16:71  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.18.239.221  netmask 255.255.240.0  broadcast 172.18.239.255
        inet6 fe80::215:5dff:feca:ba  prefixlen 64  scopeid 0x20<link>
        ether 00:15:5d:ca:00:ba  txqueuelen 1000  (Ethernet)
        RX packets 17223  bytes 12891884 (12.8 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 11407  bytes 11273398 (11.2 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 76501  bytes 85676056 (85.6 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 76501  bytes 85676056 (85.6 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 2.1: Network Configuration

Server running proof

We can utilize `the netstat` command to see running connection details such as protocol, address + port, and process ID.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ netstat -tulnp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:50294           0.0.0.0:*               LISTEN      28172/./dist/server
tcp        0      0 10.255.255.254:53       0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:46307         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:5432          0.0.0.0:*               LISTEN      -
tcp6       0      0 ::1:3350                :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::3390                 :::*                    LISTEN      -
udp        0      0 127.0.0.53:53           0.0.0.0:*                           -
udp        0      0 10.255.255.254:53       0.0.0.0:*                           -
udp        0      0 127.0.0.1:323           0.0.0.0:*                           -
udp        0      0 0.0.0.0:5353            0.0.0.0:*                           -
udp        0      0 0.0.0.0:55097           0.0.0.0:*                           -
udp6       0      0 ::1:323                 :::*                                -
udp6       0      0 :::5353                 :::*                                -
udp6       0      0 :::38933                :::*                                -
```

Figure 3.1: Network Information

User Registration

When a client sends a registration command, the server parses it and saves the user securely.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ ./dist/server
Server is listening on port 50294 ...
Client connected: 127.0.0.1:35920
Shared secret: 2
Received message: NGL8"03"PGEKQVGP8"lcfwl"lcfwl
Parsing message: LEN: 21 REGISTER: nadun nadun
Line: LEN: 21 REGISTER: nadun nadun
Command name: REGISTER, Command args: nadun nadun
Handling command: REGISTER with args: nadun nadun
Processing REGISTER command with args: nadun nadun
User 'nadun' added successfully.
User 'nadun' registered successfully.
```

Figure 4.1: user registration (server)

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option> 1
═══REGISTER═══
Enter username: nadun
Enter password: nadun
Server response: OK: 1;SID: 1042; User 'nadun' registered successfully
```

Figure 4.2: user registration (client)

When a user is successfully saved, it will be logged in to the server log file; therefore, it provides auditability.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ tail -1 server_IT24104294.log
1775969913-127.0.0.1:35920 PID: 28969 MESSAGE⇒ User registered successfully '(nadun)'
```

Figure 4.3: user registration (log)

All passwords are saved using `the SHA256` hashing algorithm with random salt.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ cat users.txt
alex:69cfc5:67034794e35758f4dcb96294b10eaf359af53688320b8dcb8a957f2d9a054b1e
admin:988cd8:3c30a17b2a2143bc9f961b0657ca650b049363dcdac58b5440d7a65fa960f2c9
smith:a94e18:cb15ec66aa80239c3e3855bb2a831bad3b5808ef1fc75df65975b25b6af9a8a5
nadun:aba04c:e29ec6d991019ab47c373f40997e1e0b2578f8ccc644a9334ec463caae767d51
```

Figure 4.4: user database

User Authentication

Users can log in to the server with a username and password; if the password is correct, the server sends a valid token for 5 minutes.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option> 0
═══LOGIN═══
Enter username: nadun
Enter password: nadun
Server response: OK: 1;SID: 1042; TOKEN: 2bab91
```

Figure 5.1: user authentication (client)

When a password is received, it will be hashed with salt, and then we compare it with the original hash. If it's valid, we send a token.

```
Client connected: 127.0.0.1:33120
Shared secret: 2
Received message: NGL8"3:"NMEKL8"lcfwl"lcfwl
Parsing message: LEN: 18 LOGIN: nadun nadun
Line: LEN: 18 LOGIN: nadun nadun
Command name: LOGIN, Command args: nadun nadun
Handling command: LOGIN with args: nadun nadun
Processing LOGIN command with args: nadun nadun
Comparing password hash: e29ec6d991019ab47c373f40997e1e0b2578f8ccc644a9334ec463caae767d51 with stored hash: e29ec6d991019ab47c373f40997e1e0b
2578f8ccc644a9334ec463caae767d51
Salt used for hashing: aba04c
Generated token for user 'nadun': 2bab91
```

Figure 5.2: user authentication (server)

This process is also logged in to the server log file.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ tail -1 server_IT24104294.log
1775969913-127.0.0.1:35920 PID: 28969 MESSAGE⇒ User registered successfully '(nadun)'
```

Figure 5.3: user authentication (log)

When a user sends an incorrect password, it will log and reject the authentication.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option> 0
═══LOGIN═══
Enter username: admin
Enter password: root
Server response: ERR: 0;SID: 1042; Error processing commands
```

Figure 5.4: user authentication failure (client)

```
Client connected: 127.0.0.1:57974
Shared secret: 2
Received message: NGL8"35"NMEKL8"cfokl"pmmv
Parsing message: LEN: 17 LOGIN: admin root
Line: LEN: 17 LOGIN: admin root
Command name: LOGIN, Command args: admin root
Handling command: LOGIN with args: admin root
Processing LOGIN command with args: admin root
Comparing password hash: 4ccf5f7c7b69245e4f4b888773d79b92acf9a18dd9ba482a6b86e106b9e9049f with stored hash: 3c30a17b2a2143bc9f961b0657ca650b
049363dcdac58b5440d7a65fa960f2c9
Salt used for hashing: 988cd8
Incorrect password for user: admin root
Error handling commands
Error handling client
```

Figure 5.5: user authentication failure (server)

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ tail -1 server_IT24104294.log
1775970171-127.0.0.1:57974 PID: 30079 MESSAGE⇒ Failed login attempt for user '(admin)'
```

Figure 5.6: user authentication failure (log)

Authorized Commands

Users can send messages with their token.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option> 0
═══LOGIN═══
Enter username: admin
Enter password: admin
Server response: OK: 1;SID: 1042; TOKEN: 1906a7
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option> 3
═══SEND MESSAGE═══
Enter token: 1906a7
Enter message: hi this message is from space
Server response: OK: 1;SID: 1042; Message received
```

Figure 6.1: message command (client)

It will be displayed on the server console. And it will be directly logged into separate log files of the user.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ cat srv/ie2102/IT24104294/
admin.log  smith.log
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ cat srv/ie2102/IT24104294/admin.log
1775970962-127.0.0.1:38012 PID: 33281 MESSAGE⇒ Received message from user '(admin)': hi this message is from space
```

Figure 6.2: message command (log)

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ ./dist/server
Server is listening on port 50294 ...
Client connected: 127.0.0.1:37456
Shared secret: 2
Received message: NGL8"3:"NMEKL8"cfokl"cfokl
Parsing message: LEN: 18 LOGIN: admin admin
Line: LEN: 18 LOGIN: admin admin
Command name: LOGIN, Command args: admin admin
Handling command: LOGIN with args: admin admin
Processing LOGIN command with args: admin admin
Comparing password hash: 3c30a17b2a2143bc9f961b0657ca650b049363dcdac58b5440d7a65fa960f2c9 with stored hash: 3c30a17b2a2143bc9f961b0657ca650b
049363dcdac58b5440d7a65fa960f2c9
Salt used for hashing: 988cd8
Generated token for user 'admin': 1906a7
Client connected: 127.0.0.1:38012
Shared secret: 2
Received message: NGL8"65"OQE8"vmigl83;24c5"jk"vjkq"ogqqceg"kq"dpmo"qrcag
Parsing message: LEN: 47 MSG: token:1906a7 hi this message is from space
Line: LEN: 47 MSG: token:1906a7 hi this message is from space
Command name: MSG, Command args: token:1906a7 hi this message is from space
Handling command: MSG with args: token:1906a7 hi this message is from space
Processing MSG command with args: token:1906a7 hi this message is from space
User 'admin' sent message: hi this message is from space
```

Figure 6.3: message command (server)

If the token is invalid, in other words, if the token is expired, the authorized commands will be rejected, and the attempt will be logged.

```
Client connected: 127.0.0.1:57798
Shared secret: 2
Received message: NGL8"1:"OQE8"vmigl8gcc`;5"jk"vjkq"kq"c"ogqqceg
Parsing message: LEN: 38 MSG: token:eaab97 hi this is a message
Line: LEN: 38 MSG: token:eaab97 hi this is a message
Command name: MSG, Command args: token:eaab97 hi this is a message
Handling command: MSG with args: token:eaab97 hi this is a message
Processing MSG command with args: token:eaab97 hi this is a message
Token expired for user 'admin'
Invalid or expired token: eaab97
Error handling commands
Error handling client
```

Figure 6.4: message command failure (server

An expired token will be removed.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ cat tokens.txt
admin:eaab97:1775929852
nadun:2bab91:1775970347
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ cat tokens.txt
nadun:2bab91:1775970347
```

Figure 6.5: Expired token removal

User log-out

Users can log out from the server. Once they do that, if there is a valid token, it will be removed; therefore, bad actors cannot utilize that token if it was compromised.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option> 2
═══LOGOUT═══
Enter username: nadun
Server response: OK: 1;SID: 1042; User 'nadun' logged out successfully
```

Figure 7,1: user log out (client)

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ ./dist/server
Server is listening on port 50294 ...
Client connected: 127.0.0.1:40640
Shared secret: 2
Received message: NGL8"31"NMEMWV8"lcfwl
Parsing message: LEN: 13 LOGOUT: nadun
Line: LEN: 13 LOGOUT: nadun
Command name: LOGOUT, Command args: nadun
Handling command: LOGOUT with args: nadun
Processing LOGOUT command with args: nadun
User 'nadun' logged out successfully.
```

Figure 7.2: user log out (server)

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ tail -5 server_IT24104294.log
1775969913-127.0.0.1:35920 PID: 28969 MESSAGE⇒ User registered successfully '(nadun)'
1775970047-127.0.0.1:33120 PID: 29619 MESSAGE⇒ User logged in successfully '(nadun)'
1775970171-127.0.0.1:57974 PID: 30079 MESSAGE⇒ Failed login attempt for user '(admin)'
1775970582-127.0.0.1:57798 PID: 30886 MESSAGE⇒ Failed MSG attempt with invalid token '(eaab97)' with message: hi this is a message
1775970725-127.0.0.1:40640 PID: 32389 MESSAGE⇒ User logged out successfully '(nadun)'
```

Figure 7.3: user log out (log)

User data storage

All users' data will be stored securely. Passwords are stored using the SHA256 hashing algorithm with a salt value.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ cat users.txt
alex:69cfc5:67034794e35758f4dcb96294b10eaf359af53688320b8dcb8a957f2d9a054b1e
admin:988cd8:3c30a17b2a2143bc9f961b0657ca650b049363dcdac58b5440d7a65fa960f2c9
smith:a94e18:cb15ec66aa80239c3e3855bb2a831bad3b5808ef1fc75df65975b25b6af9a8a5
nadun:aba04c:e29ec6d991019ab47c373f40997e1e0b2578f8ccc644a9334ec463caae767d51
```

Figure 8.1: user database

Handling Multiple Clients.

This server is able to handle multiple clients at the same time.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ ./dist/server
Server is listening on port 50294 ...
Client connected: 127.0.0.1:51374
Client connected: 127.0.0.1:33462
Client connected: 127.0.0.1:33470
Client connected: 127.0.0.1:57798
```

Figure 9.1: handling multiple clients (server)

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option>
```

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option>
```

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option>
```

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ python3 client_4294.py
Choose an option:
0: LOGIN
1: REGISTER
2: LOGOUT
3: SEND MESSAGE
option>
```

```
[0] 0:bash*                                                    "DESKTOP-CG383E4" 11:34 12-Apr-26
│                  ├─SessionLeader──Relay(28002)─┬─bash──sudo──sudo──su──bash──tmux: client
│                  │                             └─tmux: server──4*[bash──python3]
│                  ├─init──{init}
│                  ├─login──bash
│                  └─{init-systemd(Ub}
```

Figure 9.2: creating multiple clients using tmux

```
├─init-systemd(Ub─┬─SessionLeader──Relay(24508)──bash──sudo──sudo──su──bash──server──4*[server]
│                  ├─SessionLeader──Relay(27543)──bash──sudo──sudo──su──bash──pstree
│                  ├─SessionLeader──Relay(28002)─┬─bash──sudo──sudo──su──bash──tmux: client
│                  │                             └─tmux: server──4*[bash──python3]
│                  ├─init──{init}
│                  ├─login──bash
│                  └─{init-systemd(Ub}
```

Figure 9.3: pstree

Build server

We can use **`Makefile`** to build the server executable with ease.

```
nadun@DESKTOP-CG383E4:/mnt/d/Other/University/Projects/SecureServer$ make -f Makefile_4294
═══════════════════════════════════════════════
 ____
/ ___|
| (__   __  __  __  __ __ __   ____ _ __
\__ \ / _ V  __| / __|/ _ \ '__\ \ / / _ \ '__|
___) |  __/ (__  \__ \  __/ |   \ V /  __/ |
|____/\___|\___| |___/\___|_|    \_/ \___|_|
═══════════════════════════════════════════════
Building server and client ...
Compilation complete.
```

Figure 10.1: Build server

Figure Table

Figure 1.1: Hostname 3
Figure 2.1: Network Configuration 3
Figure 3.1: Network Information 4
Figure 4.2: user registration (client) 4
Figure 5.1: user authentication (client) 5
Figure 6.1: message command (client) 6
Figure 7.2: user log out (server) 8
Figure 8.1: user database 9
Figure 9.2: creating multiple clients using tmux 10
Figure 10.1: Build server 11